

Project Specification: Aegis-K8s Smart Traffic Router

1. Project Description

The Aegis-K8s Smart Traffic Router is a high-level DevOps engineering project designed to demonstrate mastery over core Kubernetes primitives. The primary objective is to build a resilient, self-healing microservices architecture that manages complex traffic patterns without the use of external service meshes like Istio.

The system simulates a production-grade environment by deploying a backend application across three distinct lifecycle stages: Stable (v1), Faulty (v2), and Optimized (v3). By utilizing native Kubernetes objects—including Ingress Controllers, Horizontal Pod Autoscalers (HPA), and ConfigMaps—this project showcases intelligent traffic distribution, automated failure recovery, and real-time dynamic configuration.

1.1 Project Scope & Resource Constraints

- **Zero-Cost Infrastructure:** This project is built using strictly free or provided resources. All deployments are targeted for local clusters (Minikube/Kind) or AWS Free Tier instances using existing credits.
- **No Paid Services:** The architecture specifically avoids paid managed databases, third-party monitoring SaaS, or premium load balancers. All logic (routing, health, scaling) is handled by the Kubernetes control plane itself.

2. Key Features & Functional Requirements

A. Tri-Deployment Backend

- **Versioned Deployments:** Three separate deployments, each running 3 replicas.
 - **v1 (Stable):** Reliable, standard API responses.
 - **v2 (Faulty):** Integrated "Chaos API" to simulate crashes and errors.
 - **v3 (Optimized):** Tuned for low-latency performance.
- **Metadata Transparency:** Every response must include the specific Pod Name and Version ID to verify distribution logic.

B. Intelligent Traffic Routing

- **Weighted Canary Splitting:** The Nginx Ingress Controller must be configured to distribute traffic at a precise ratio: 70% to v1, 10% to v2, and 20% to v3.
- **Session Affinity:** Implementation of "sticky sessions" to ensure a consistent user experience during version transitions.

C. Resilience & Self-Healing

- **Liveness & Readiness Probes:** Native K8s probes to automatically restart frozen containers and prune unhealthy pods from the traffic pool.
- **Chaos Suite:** Dedicated endpoints (`/simulate/crash`, `/simulate/lag`, `/simulate/load`) to trigger manual system failures for testing.

D. Scaling & Dynamic Configuration

- **Horizontal Pod Autoscaling (HPA):** Dynamic scaling from 3 to 10 replicas based on a 50% CPU utilization threshold.
- **Hot-Reload ConfigMaps:** Use ConfigMaps for feature flags. The application must detect updates and change behavior without a pod restart.

3. Accelerated 10-Day Implementation & Verification Plan

To achieve 10/10 deliverability without dropping features, this plan introduces **Strict Timeboxing** and **Technical Guardrails**. You must use a lightweight language (e.g., Python FastAPI or Node.js Express) and leverage Minikube's built-in addons to avoid environment configuration hell.

Phase	Day	Milestone Task & Technical Guardrail	Verification Method (Definition of Done)
Phase 1: The Core	1	Env & API Design: Setup Minikube. Build base API. <i>Guardrail:</i> Do not build a database. Hardcode JSON responses.	Run <code>curl /health</code> and receive a 200 OK with version metadata.

	2	Chaos Engineering: Implement crash/lag/load endpoints. <i>Guardrail:</i> Use simple <code>os._exit(1)</code> for crash and <code>time.sleep(10)</code> for lag.	Trigger <code>/simulate/lag</code> and verify response delay via <code>time curl</code> .
	3	Containerization: Write Dockerfiles; build and tag images. <i>Guardrail:</i> Use Alpine-based images to guarantee fast local pulls.	<code>docker images</code> shows all 3 versions under 100MB each.
Phase 2: Orchestration	4	K8s Deployments: Apply YAMLS for all 3 versions. <i>Guardrail:</i> Use <code>minikube</code> image load instead of an external Docker registry to save time.	<code>kubect! get pods</code> shows 9 running pods across 3 deployments.
	5	Ingress & Routing: Install Nginx Ingress; apply 70/10/20 weights.	Run 100-request loop; verify ~70 hits to v1 via <code>logs/script</code> .

		<i>Guardrail:</i> Run <code>minikube addons enable ingress</code> first. Use <code>nginx.ingress.kubernetes.io/canary</code> annotations.	
	6	Affinity & Probes: Enable sticky sessions and health probes. <i>Guardrail:</i> Set <code>initialDelaySeconds: 5</code> on probes to prevent crash-loops during startup.	Crash a v2 pod via API; verify K8s restarts it automatically.
Phase 3: Advanced Logic	7	Scaling & Config: Setup HPA and ConfigMap hot-reload. <i>Guardrail:</i> Run <code>minikube addons enable metrics-server</code> . Mount ConfigMap as a volume; read the file on every API request.	<code>kubectl describe configmap</code> update reflects in app without pod restart.
	8	Load Testing: Stress the system using the <code>/load</code> endpoint. <i>Guardrail:</i> Ensure your Pods have <code>resources.requests.cpu</code> defined, or HPA will fail.	Observe <code>kubectl get hpa</code> scaling replicas from 3 to 10.

	9	Observability: Build a simple Bash test script. <i>Guardrail:</i> Do not build a complex UI. Use a simple terminal output summarizing 100 curl requests.	Final "Test Script" output confirms all features work in one run.
	10	Final Delivery: Cleanup YAMLs, finalize README. <i>Guardrail:</i> Include exact copy-paste <code>kubectl</code> commands in your README.	GitHub Repo is public/shared with all manifests and docs.

4. Deliverables for Submission

All deliverables must be provided in a single repository, structured strictly as follows to ensure rapid recruiter/client review:

- **Kubernetes Manifests:** A `/k8s` directory containing clean, commented YAML files for:
 - Deployments (v1, v2, v3)
 - Services (ClusterIP)
 - Ingress (Nginx Annotations for weights/affinity)
 - HPA & ConfigMaps
- **Docker Assets:** A `/docker` directory containing the application source code and version-controlled Dockerfiles.
- **Validation Reports:** A `/tests` directory containing a Bash/Python script and a text file capturing the terminal output of the 70/10/20 traffic split and HPA scaling events.
- **Local Setup Guide:** A professional `README.md` explaining how to deploy the entire stack from scratch on a clean Minikube or AWS instance using copy-paste commands.